

MEDIATEK

MT8676 Android Audio User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Jianxin Xu	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	3
List of Tables	4
1 Audio	5
1.1 Overview	5
1.1.1 Brief Summary of Audio Feature	5
1.1.1.1 Audio Feature and Blocks	5
1.1.1.2 Audio Format Support	9
1.2 Audio SW Architecture	12
1.2.1 Audio HAL	13
1.2.1.1 Audio HAL Playback	14
1.2.1.2 Audio HAL Record	15
1.2.2 ALSA Driver Architecture Overview	16
1.2.3 DAPM and DPCM Overview	17
1.3 AAOS Volume Control	18
1.3.1 Volume Config in XML	19
1.3.2 Volume Command Customization	20
1.4 External HW Devices	20
1.4.1 MT8676 I2S Capability Support	20
1.4.1.1 I2S Application	23
1.5 Mediatek Aurisys and Open DSP	23
1.5.1 Overview	23
1.5.1.1 Aurisys Structure	24
1.5.2 Data Path Customization Guideline	25
1.6 Kernel DTS Configuration	25
1.6.1 Audio Path Configuration(AFE HW)	27
1.6.2 Audio Path Configuration (INT ADSP)	28
1.6.2.1 XML Configuration Instructions	28
1.6.2.2 mixer_target & mixer_source	29
1.6.2.3 Task Attribute	29
1.6.2.4 Configuration Example	30
1.7 Summary	31
1.8 Appendix	31
1.8.1 MTK MOL	31
Exhibit 1 Terms and Conditions	32

List of Figures

Figure 1-1. MT8676 audio blocks	7
Figure 1-2. MT6368 audio blocks	7
Figure 1-3. Audio software architecture	13

Figure 1-4. Audio software data path.....	13
Figure 1-5. Audio HAL architecture.....	14
Figure 1-6. Playback architecture.....	15
Figure 1-7. Record architecture.....	16
Figure 1-8. ALSA pcm interface.....	17
Figure 1-9. Audio volume control.....	19
Figure 1-10. Audio volume grope config.....	19
Figure 1-11. Devices gain config.....	20
Figure 1-12. Aurisys concept.....	24
Figure 1-13. Aurisys structure.....	25
Figure 1-14. DTS feature list.....	26
Figure 1-15. memif define.....	26
Figure 1-16. Audio hal pcm config.....	27
Figure 1-17. ADSP architecture.....	27
Figure 1-18. Audio path for playing music.....	27
Figure 1-19. Playback_3 to PMIC.....	28
Figure 1-20. ADSP architecture.....	28
Figure 1-21. dsp task config.....	31

List of Tables

Table 1-1. GPIO function.....	21
Table 1-2. I2S application.....	23

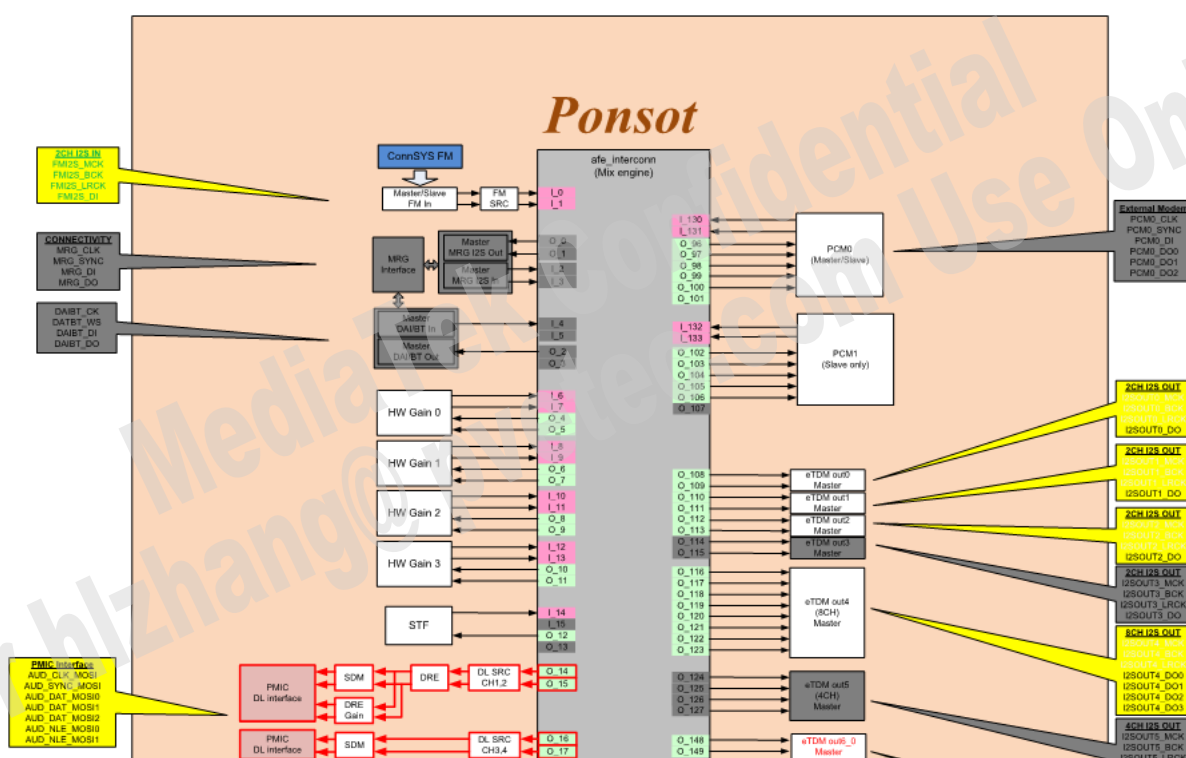
1 Audio

1.1 Overview

This document introduces audio features such as volume control, audio frameworks, DSP audio configuration, etc. First, an overview of supported features and audio block diagram are provided. Then, the AAOS volume control is introduced. Finally, the Aurisys sound framework and DSP audio mixing configuration are introduced.

1.1.1 Brief Summary of Audio Feature

1.1.1.1 Audio Feature and Blocks





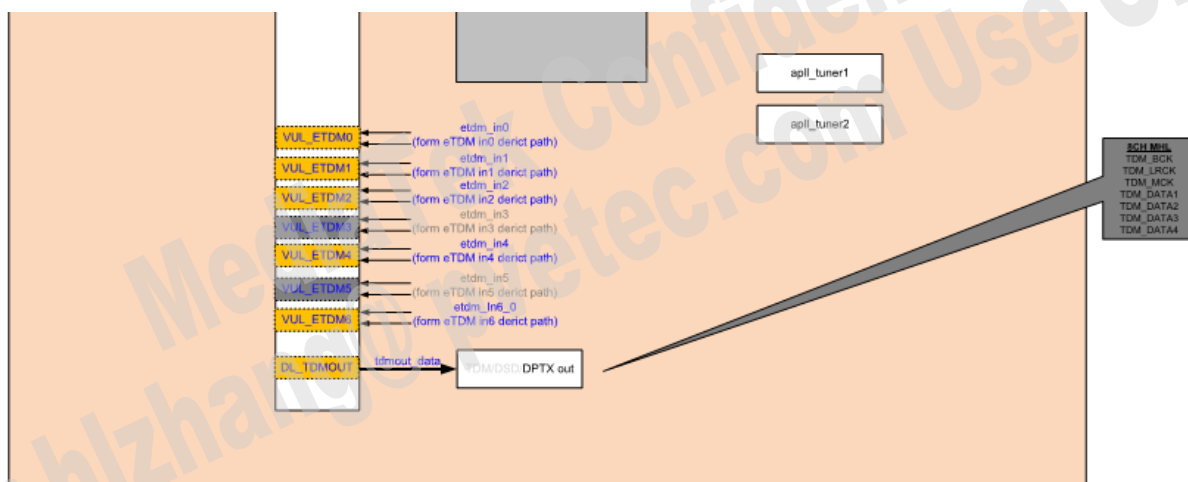


Figure 1-1. MT8676 audio blocks

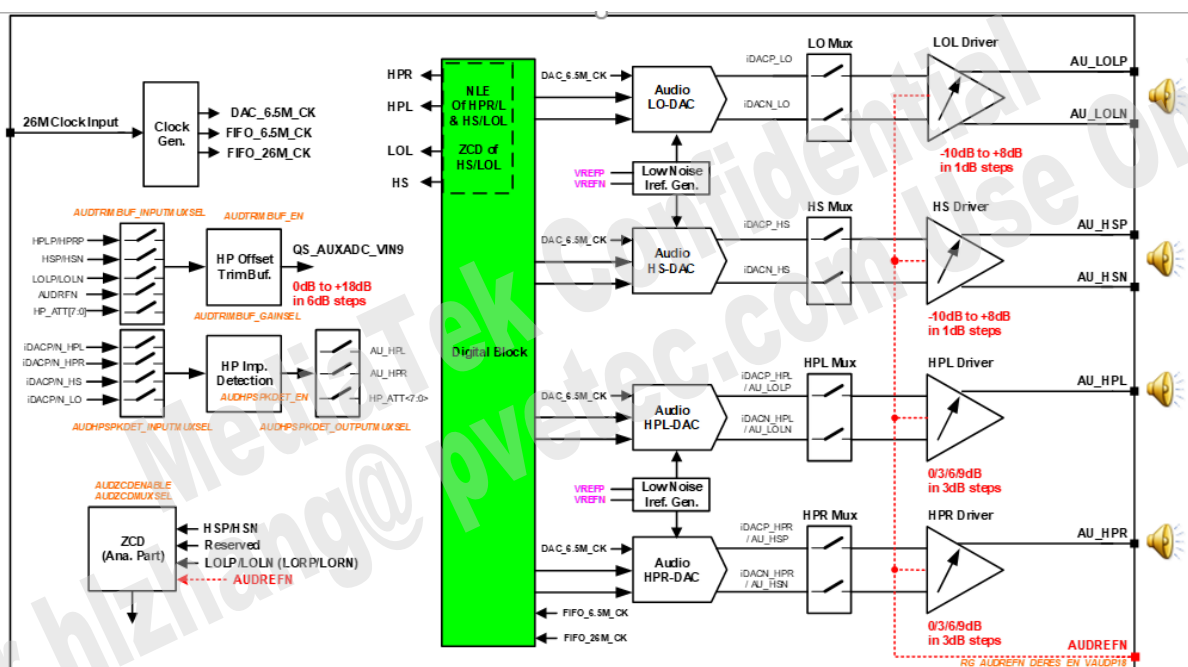


Figure 1-2. MT6368 audio blocks

- MT8676
 - MediaTek's proprietary audio interface connects to PMIC MT6368
 - Audio Playback with MTK PMIC – MT6368
 - Supports 8kHz, 11.025kHz, 12kHz, 16kHz, 22.05kHz, 24kHz, 32kHz, 44.1kHz, 48kHz, 96kHz, 192kHz, 384kHz sample rates
 - Audio Recording with MTK PMIC – MT6368
 - Supports 8kHz, 16kHz, 32kHz, 48kHz, 96kHz, and 192kHz sample rates
 - Internal high resolution , high flexible HW gain in AFE interconnection.....(1)
 - 11 sets Inter-IC Sound Interface (I2S)
 - Master Output *5 (one is 8 ch)(4)
 - Master Input * 5 (one is 8 ch)(4)
 - Slave input(with SRC)(3)

- Master Mode supports 8, 11.025, 12, 16, 22.05, 24, 32, 44.1, 48, 88, 96, 176 and 192 kHz sampling rates
- Slave Mode supports 8, 11.025, 12, 16, 22.05, 24, 32, 44.1, and 48 kHz sampling rates
- 16/32-bit bus width support
- Philip standard and Left just
- Sets up twosets of Pulse-Code Modulation (PCM) interfaces
 - Slave PCM for internal Modem*1.....(5)
 - 16/24-bit stereo data format support
- Sets up twosets of Enhanced Time Division Multiplexing (TDM) interfaces
 - Master Output *1 (up to 8 ch)(8)
- Audio codecs
 - MP3, AAC, AAC+, AMR-NB, AMR-WB, OGG, WAV, APE
- Audio Post-Processing
 - BesLoudness
 - ACF
- 3rd party support (need contact with 3rd party by customer)
 - Dolby mobile
- MT6368
 - MediaTek's proprietary audio interface connects to MT8676
 - TX brief spec

		MT6897 + MT6368
DOWNLINK PATH		
# of DACs		3
# of analog outputs		4
Rx concurrency		Up to 2
Max. sampling rate/Bit resolution		192 kHz/24-bit
Output buffer type		DC coupled – ClassAB
Audio buffer	Single-ended (S.E.)/Differential (Diff.)	S.E. *2
	Max. output swing @ 0.1% THD+N	1 Vrms (S.E.) @ 32Ω
	Gain range (dB)	-10~ +8
	Gain per step	1 dB/step
Voice Buffer	Output type	Diff. *1
	Max. output swing @ 1% THD+N	1.87 Vrms-vd @ 32Ω
	Gain range (dB)	-10 ~ +8
	Gain per step	1 dB/step
Line out buffer	Single/Differential	Diff. *1
	Max. output swing	1.88 Vrms-Vd @ 600Ω
SPK amp	Output type	X

- RX brief spec

MT6897 + MT6368		
UPLINK PATH		
# of ADCs		3
# of analog inputs		4
Tx concurrency		Up to 4
Max sampling rate/Bit resolution		192 kHz/24-bit
Bias circuit		DCC mode
Gain	Gain range (dB)	0~45
	Gain per step	1 dB
# of MICBIAS outputs		3
MICBIAS voltage (V)		1.7 ~ 2.1 & 2.5 ~ 2.7, 0.1per step
MICBIAS current (mA)		Up to 3
# of digital MICs		O (3MICs on MT6368 + 6MICs on MT6897)
Digital MIC CLK Rate (Hz)	26 MHz CLK source	4.33M/3.25M/1.625M/812.5K/406.25K
	32 kHz CLK source	1.625M/812.5K/541.67K
OTHERS		
Accessory detection	UL DCC mode	HP_EINT (MT6368, EINT)
G (Ground) detection		O
Voice wake-up (VoW)		O (1ch or 2ch)
ADSP		O (HiFi3)
SCP		O (RV55NN)

1.1.1.2 Audio Format Support

- Playback
 - AAC/HE-AAC v1/HE-AAC v2
 - Android Orientated Support/Not use Mediatek IP
 - 8 kHz ~ 96 kHz; 8 kbps ~ 320 kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR/CBR
 - File Extension: .aac (ADTS, ADIF), .m4a, .mp4, .3gp, .ts
 - Profile: 1) LC, HEAAC V1, V2; 2) LD, ELD
 - LC, 48kHz, 128kbps, stereo → MCPS = 11; HEV1, 22.05kHz, 128kbps, stereo → MCPS = 31; HEV2, 22.05kHz, 32kbps, stereo → MCPS = 40; ELD, 44.1kHz, 128kbps MCPS = 13
 - AMR
 - Android Orientated Support/Use Mediatek IP
 - 8 kHz, 4.75kbps~12.2kbps
 - Mono Support

- Bitrate mode: CBR
- File Extension: .amr
- 8kHz, 12.2kbps, mono -> Mediaserver:50 MCPS, AMR Dec: 7 MCPS
- AWB
 - Android Orientated Support/Use Mediatek IP
 - 16 kHz, 6.6kbps~23.85kbps
 - Mono Support
 - Bitrate mode: CBR
 - File Extension: .awb
 - 16kHz, 23.85kbps, mono -> Mediaserver:60 MCPS, AMR Dec: 18 MCPS
- MIDI
 - Android Orientated Support / Not use Mediatek IP
 - 22.05kHz
 - Stereo Support
 - File Extension: .mid, .midi, .smf, .rtttl, .xmf, .rtx, .ota, .imy
- MP2
 - Android Not Support / Use Mediatek IP
 - 8 kHz ~ 48 kHz; 8 kbps ~ 320 kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR/CBR
 - File Extension: .mp2
 - Profile: MPEG1-Layer2, MPEG2-Layer2, MPEG2.5-Layer2
 - MPEG1, 48kHz, 256kbps, stereo → 11 MCPS
- MP3
 - Android Orientated Support / Use Mediatek IP
 - 8 kHz ~ 48 kHz; 8 kbps ~ 320 kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR/CBR
 - File Extension: .mp3
 - Profile: MPEG1-Layer3, MPEG2-Layer3, MPEG2.5-Layer3
 - MPEG1, 48kHz, 256kbps, stereo → 15 MCPS
- OGG VORBIS
 - Android Orientated Support / Use Mediatek IP
 - 8 kHz ~ 192 kHz; 10 kbps ~ 320 kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR
 - File Extensions: .ogg, .oga
 - 48kHz, 250bps, stereo → MCPS = 25
- WAV (ADPCM)
 - Android Not Support / Use Mediatek IP
 - 8 kHz ~ 192 kHz; 4 kbps ~ 192 kbps
 - Bitrate Mode: CBR
 - File Extension: .wav
 - Profile: DVI/MA ADPCM; MS ADPCM

- DVI/MA 48kHz, stereo → 3.44 MCPS; DVI/MA 192 kHz, stereo → 13.6 MCPS; MS, 48kHz, stereo → 4.17 MCPS; MS, 192 kHz, stereo → 16.3 MCPS
- WAV (Alaw/Ulaw)
 - Android Orientated Support / Use Mediatek IP
 - 6 kHz ~ 96 kHz; 48 kbps ~ 3072 kbps
 - Mono//Stereo Support
 - File Extension: .wav
- WAV (Raw)
 - Android Orientated Support / Use Mediatek IP
 - 6 kHz ~ 96 kHz; 48 kbps ~ 3072 kbps
 - 1 channel ~ 8 channels support
 - File Extension: .wav
- APE
 - Android Not Support / Use Mediatek IP
 - 6 kHz ~ 96 kHz; 29 kbps ~ 836 kbps
 - Mono//Stereo Support
 - Bitrate Mode: VBR
 - File Extension: .ape
 - Profile: fast, normal, high, extra high
 - APE normal compress type, 44.1kHz, 675 kbps, stereo → MCPS = 50
- WMA (Need license)
 - Android Not Support / Use Mediatek IP
 - 8 kHz ~ 48 kHz; 5 kbps ~ 320 kbps
 - Mono//Stereo Support
 - Bitrate Mode: VBR
 - File Extension: .wma
 - Profile: WMA v1; WMA v2
 - 32 kHz, 22kbps, stereo → MCPS = 22.57
- FLAC
 - Android Orientated Support / Use Mediatek IP
 - 8 kHz ~ 48 kHz; 87 kbps ~ 396 kbps
 - Mono//Stereo Support
 - Bitrate Mode: VBR
 - File Extension: .flac
 - 44.1 kHz, 745kbps, level5, stereo → MCPS = 10
- Record
 - AAC
 - Android Orientated Support / Not use Mediatek IP
 - 8 kHz ~ 48 kHz; 8kbps~160kbps
 - Mono//Stereo Support
 - Bitrate Mode: CBR
 - File Extension: .3gp, .aac
 - Profile: Low Complexity; High Efficiency; Enhanced Latency Delay
 - 48kHz, 128kbps, stereo (Low Complexity) → MCPS = 32; 48kHz, 128kbps, stereo (High Efficiency) → MCPS = 75; 48kHz, 128kbps, stereo (Low Latency Delay) → MCPS = 40;

- AMR
 - Android Orientated Support / Use Mediatek IP
 - 8 kHz; 4.75kbps~12.2kbps
 - Mono Support
 - Bitrate Mode: CBR
 - File Extensions: .3gp, .amr
 - 8kHz, 12.2kbps, mono → MCPS = 39
- AWB
 - Android Orientated Support / Use Mediatek IP
 - 16 kHz; 6.60kbps~23.85kbps
 - Mono Support
 - Bitrate Mode: CBR
 - File Extension: .3gp, .awb
 - 16kHz, 23.85 kbps, mono → MCPS = 77
- OGG
 - Android Not Support / Use Mediatek IP
 - 8 kHz~ 48 kHz; 31.98kbps~202.96kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR
 - File Extension: ..ogg
 - 48kHz, 128 kbps, stereo → MCPS = 40
- ADPCM
 - Android Not Support / Use Mediatek IP
 - 8 kHz~ 48 kHz; 4kbps~192kbps
 - Mono/Stereo Support
 - Bitrate Mode: CBR
 - File Extension: ..wav
 - 16 kHz, stereo, DIV/IMA → MCPS = 2.88; 48 kHz, stereo, DIV/MA → MCPS = 8.5; 16 kHz, stereo, MS → MCPS = 5.08; 48 kHz, stereo, MS → MCPS = 14.97

1.2 Audio SW Architecture

The audio software architecture and data flow are shown as [Figure 1-3](#) and [Figure 1-4](#) respectively. Information about the software components and data path can be found in these figures.

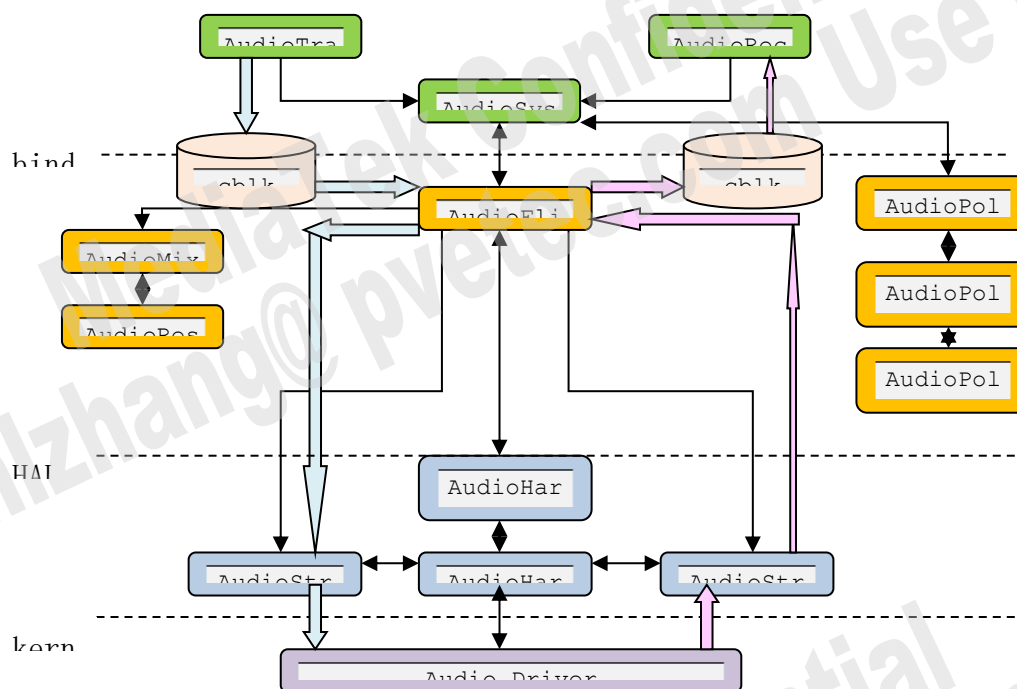


Figure 1-3. Audio software architecture

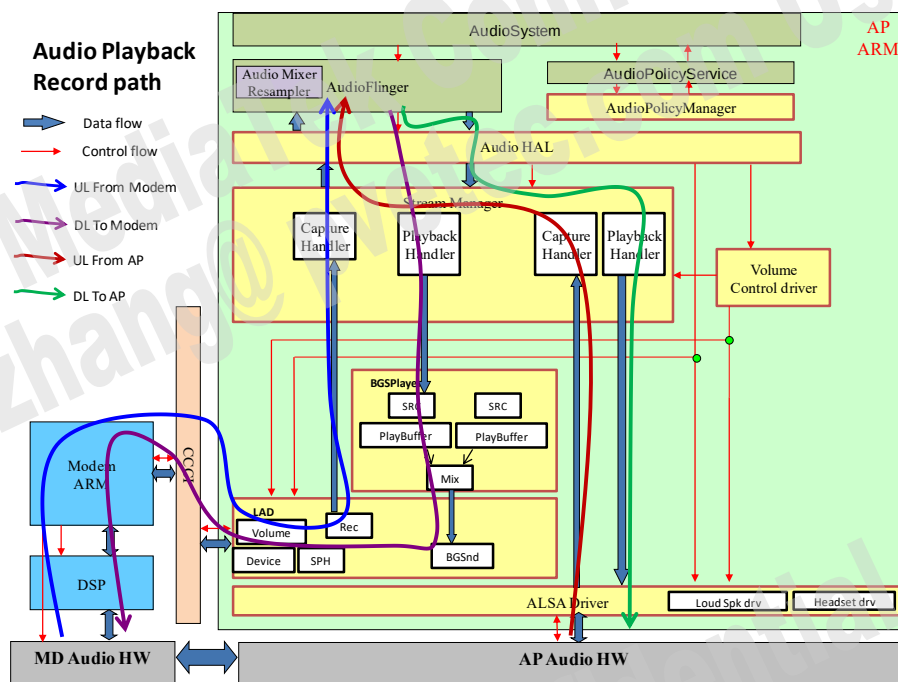


Figure 1-4. Audio software data path

1.2.1 Audio HAL

Figure 1-5 shows the architecture of the audio HAL.

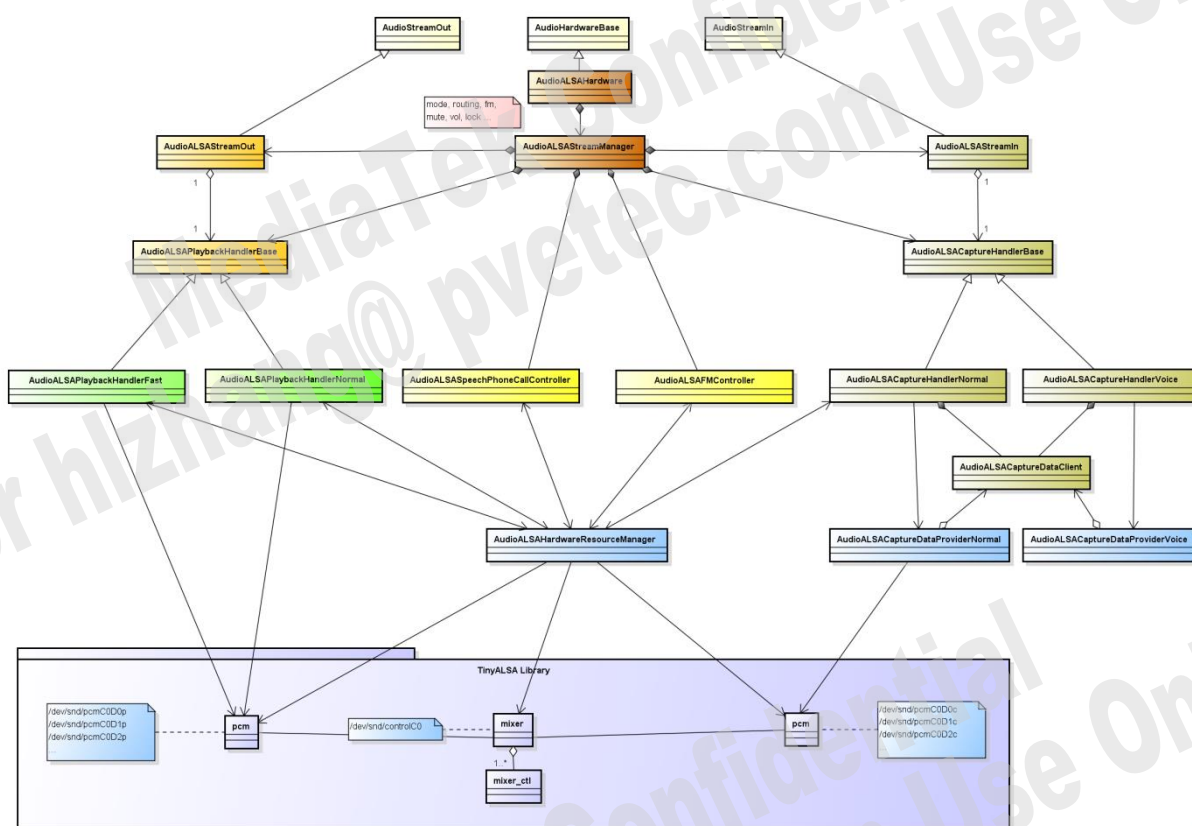


Figure 1-5. Audio HAL architecture

The top control interfaces of HAL consist of `AudioALSAHardware`, `AudioALSAStrOut`, and `AudioALSAStrIn`. These classes all utilize `AudioALSAStrManager` for controlling audio modes and open/close, input/output streams.

Consequently, AudioALSAStreamManager will contain all audio environment information such as mode, routing, volume, and mute details.

1.2.1.1 Audio HAL Playback

AudioALSAStrStreamOut: A class to do write/standby/routing operation but:

- Do not implement detail in itself but using a Playback Handler to bypass open/close/routing/write operations
- Different scenarios use different types of playback handlers
- Stream Out will call createPlaybackHandler() in StreamManager to acquire a pointer of playback handler when 1st write().
- Also, call destroyPlaybackHandler() when standby().

HW registers should not be set in the playback handler, but TinyALSA Library should be used for the call.

- `pcm_open() / pcm_close()`
 - To control AFE hw path, memory setting
- `mixer_open()/mixer_close()`
 - To get various types of mixer control like, `Speaker_Amp_Switch`, `Voice_Amp_Switch`, `Audio_Amp_R_Switch`, and `Audio_Amp_L_Switch` to control codec driver.
- `pcm_write()`
 - To write PCM data to SRAM/DRAM

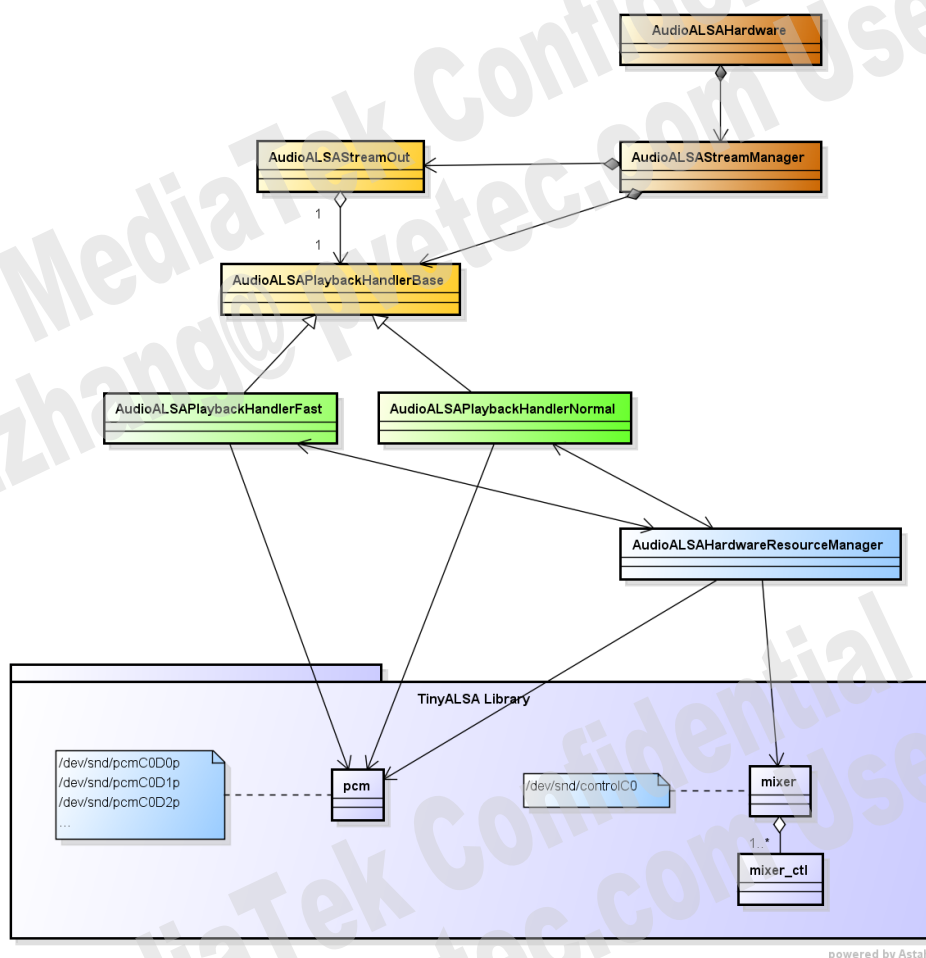


Figure 1-6. Playback architecture

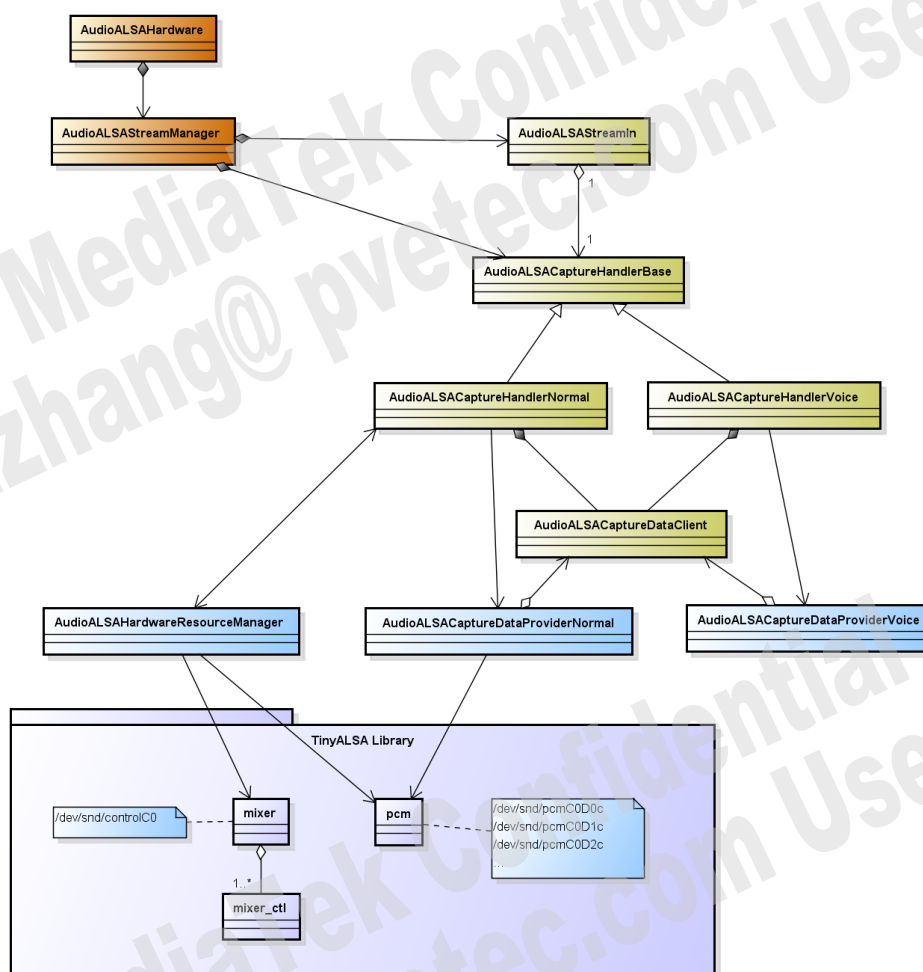
1.2.1.2 Audio HAL Record

AudioALSAStreamIn: A class to do read/standby/routing operation but:

- Do not implement detail in itself but using a Capture Handler to bypass open/close/routing/read operations
- Different scenarios use different types of capture handlers
- Stream In will call createCaptureHandler() in StreamManager to acquire a pointer to the capture handler when 1st read().
- Also, destroyCaptureHandler() when standby()

HW registers should not be set in the capture handler; instead, use the TinyALSA Library for calling.

- pcm_open() /pcm_close()
 - To control AFE hw path, memory setting
- mixer_open()/mixer_close()
 - To get various types of mixer control like, Audio_ADC_1_Switch, Audio_ADC_2_Switch, Audio_Preamplifier1_Switch, Audio_Preamplifier2_Switch to control codec driver.
- pcm_read()
 - To read PCM data into SRAM/DRAM



powered by Astah

Figure 1-7. Record architecture

1.2.2 ALSA Driver Architecture Overview

The Advanced Linux Sound Architecture (ALSA) provides audio and MIDI functionality to the Linux operating system. ALSA has the following significant features:

- Efficient support for all types of audio interfaces, from consumer sound cards to professional multichannel audio interfaces.
- Fully modularized sound drivers.
- SMP and thread-safe design.
- User space library (alsa-lib) to simplify application programming and provide higher level functionality.
- Support for the older Open Sound System (OSS) API, providing binary compatibility for most OSS programs.

The MTK SoC ALSA driver provides a smartphone PCM interface for the User space HAL.

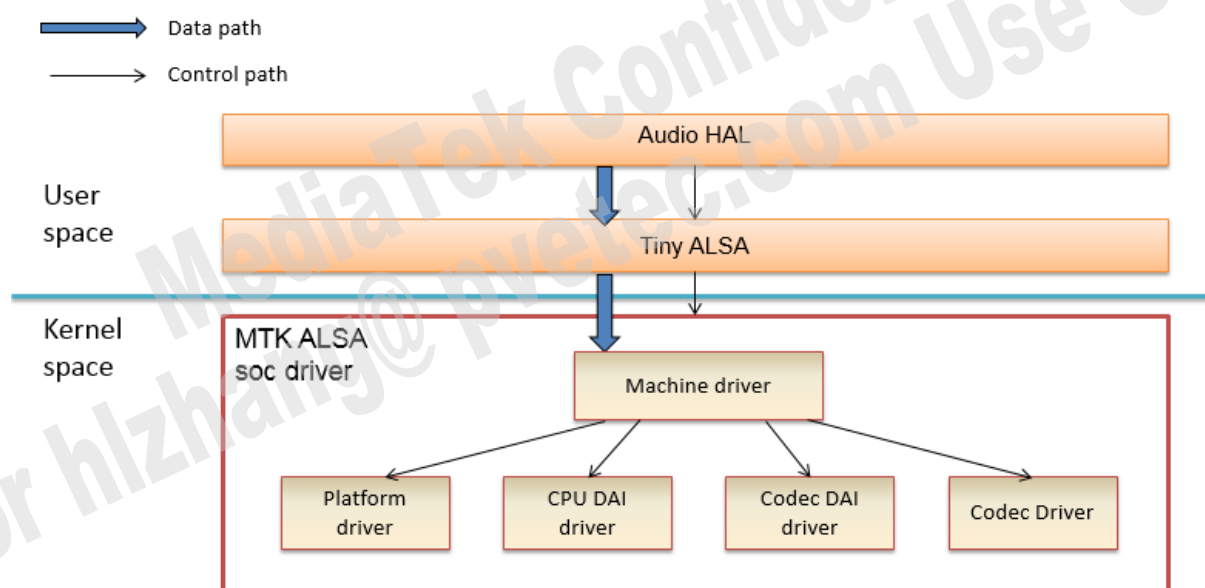


Figure 1-8. ALSA pcm interface

1.2.3 DAPM and DPCM Overview

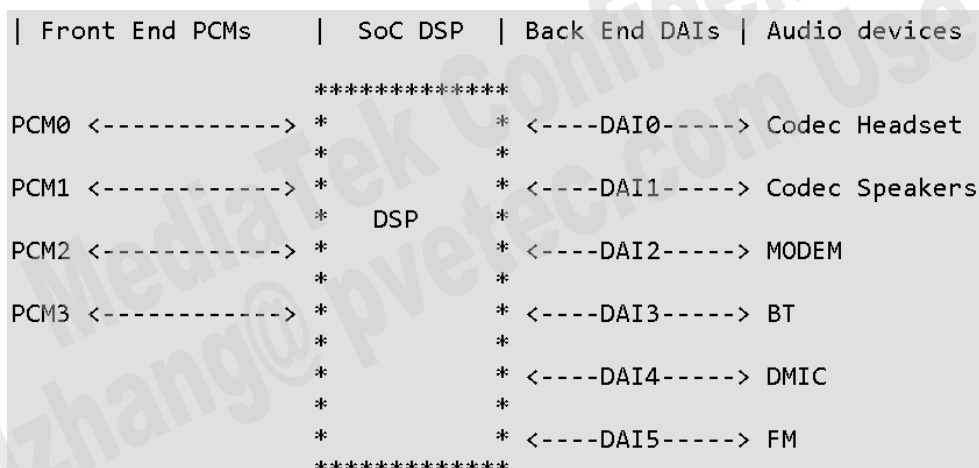
Dynamic Audio Power Management (DAPM) is designed to allow portable Linux devices to use the minimum amount of power within the audio subsystem at all times. It is independent of other Kernel PM and as such, can easily co-exist with the other PM systems.

DAPM is also completely transparent to all user space applications as all power switching is done within the ASoC core. No code changes or recompiling are required for user space applications. DAPM makes power switching decisions based upon any audio stream (capture/playback) activity and audio mixer settings within the device.

All DAPM power switching decisions are made automatically by consulting an audio routing map of the whole machine. This map is specific to each machine and consists of the interconnections between every audio component (including internal codec components). All audio components that affect power are called widgets hereafter. For more detail related, please refer: <https://www.kernel.org/doc/html/v6.1/sound/soc/dpcm.html>

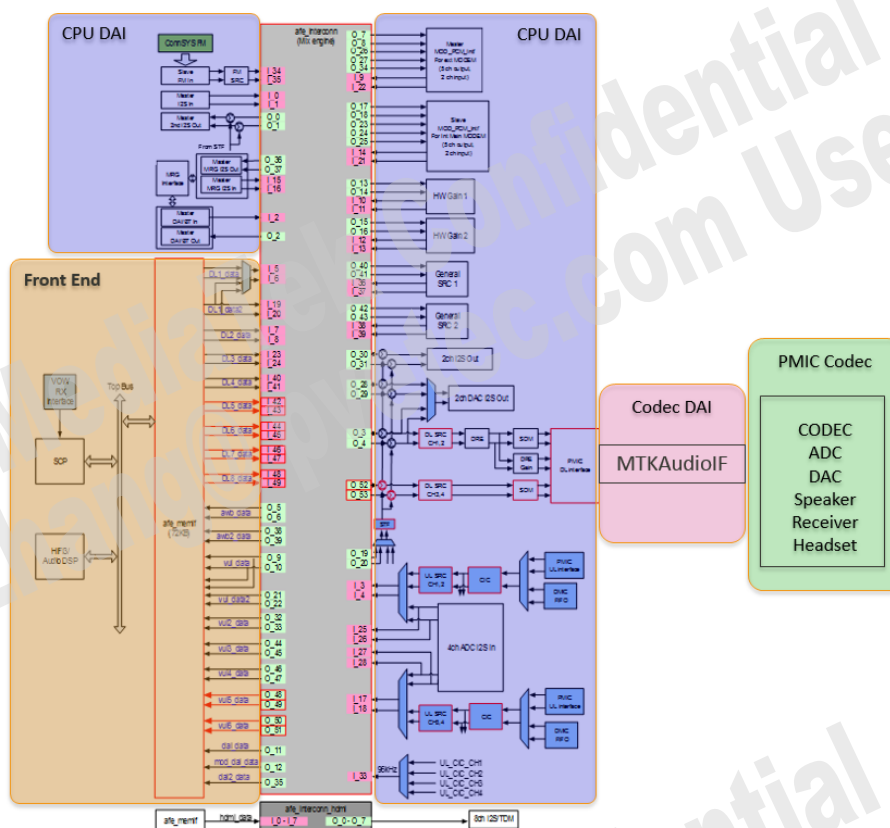
Audio driver is based on the standard DAPM and DPCM (Dynamic PCM) architecture to control PCM open/close, power on/off, and clock on/off. DPCM separates PCM into Front End/Back End PCM.

- FE PCM: Control of DMA (MEMMEMIF)
- BE PCM: Control of DAI (ADDA, PCM IF, I2S)



For more details, please refer to: <https://www.kernel.org/doc/html/v6.1/sound/soc/dpcm.html>

The following is MT8676 FE(memif) and BE(ADDA/I2S/PCM/Hostless/..) block diagram.



1.3 AAOS Volume Control

As Google defined, AAOS implementations use a hardware amplifier to control volume instead of a software mixer, so it is recommended to use smartpa or other hardware gain to adjust the volume. MTK provides interfaces and software path for adjusting the volume in the internal adsp as shown in [Error! Reference source not found.](#)

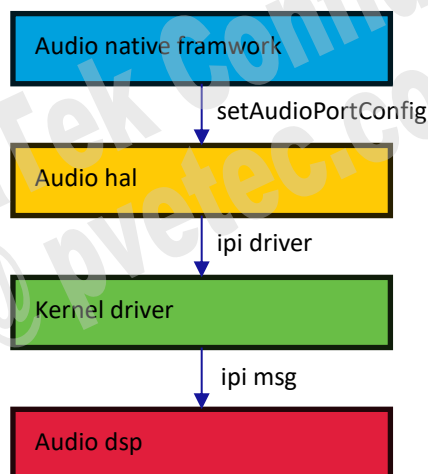


Figure 1-9. Audio volume control

1.3.1 Volume Config in XML

To config volume, we need modify both car_audio_configuration.xml and audio_policy_configuration.xml.

Car_audio_configuration.xml defines volume groups which manage the volumes for a collection of devices within an audio zone. For each volume group, the volume can be controlled independently. The resulting gains are configured on the associated devices to be applied by the vehicle's amplifier. Each volume group should contain one or more output devices with associated addresses. Addresses should correspond to the output devices defined in audio_policy_configuration.xml.

```

<carAudioConfiguration version="3">
  <zones>
    <zone name="primary zone" isPrimary="true" audioZoneId="0" occupantZoneId="0">
      <zoneConfigs>
        <zoneConfig name="primary zone config" isDefault="true">
          <volumeGroups>
            <group>
              <device address="bus0_media_out">
                <context context="music"/>
              </device>
              <device address="bus3_call_ring_out">
                <context context="call_ring"/>
              </device>
              <device address="bus6_notification_out">
                <context context="notification"/>
              </device>
              <device address="bus7_system_sound_out">
                <context context="system_sound"/>
                <context context="emergency"/>
                <context context="safety"/>
                <context context="vehicle_status"/>
                <context context="announcement"/>
              </device>
            </group>
          </volumeGroups>
        </zoneConfig>
      </zoneConfigs>
    </zone>
  </zones>
</carAudioConfiguration>
  
```

Figure 1-10. Audio volume grope config

Audio_policy_configuration.xml defines the volume group gains. Each volume group has minimum, maximum, and default gain values as well as a step size based on values configured in audio_policy_configuration.xml for the devices associated with the volume group.

```

<devicePort tagName="bus0_media_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus0_media_out">
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="0" maxValueMB="300" defaultValueMB="300" stepValueMB="30"/>
  </gains>
</devicePort>
<devicePort tagName="bus1_navigation_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus1_navigation_out">
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="0" maxValueMB="300" defaultValueMB="300" stepValueMB="30"/>
  </gains>
</devicePort>

```

Figure 1-11. Devices gain config

We provide minimum 0 to maximum 300 gain level, and gain will be mapped to float value 0~1 with logarithmic conversion function and apply to digital audio stream.

1.3.2 Volume Command Customization

Volume command is sent from audio flinger by `setAudioPortConfig` and process command in `AudioALSAPlaybackHandlerXXXX::setVolume`. Customers can modify the function implementation for customization such as adjust volume curve and set kcontrol.

1.4 External HW Devices

1.4.1 MT8676 I2S Capability Support

I2S capability interface description:

- MTK Audio Interface
 - **MTK-specific** interface for MT6368
- I2S0 – ADC/DAC interface
 - Proposed for any audio interface
 - Support standard I2S in **Master input**
 - Capability up to 192k/32-bit input.
 - Capability up to 384k/32-bit output.
- I2S1 – ADC/DAC interface
 - Support standard I2S in **Master output**
 - Maximum of 1 data wire for support of 2 channels
 - Capability up to 192k/32-bit input.
 - Capability up to 384k/32-bit output.
- I2S2 – ADC/DAC interface
 - Support the standard I2S in the Master input.
 - Maximum of 2 data wires for 4-channel support
 - Capability up to 192k/32-bit input.
 - Capability for up to 384k/32-bit output.
- I2S4 – ADC/DAC interface
 - Support standard I2S in **Master output**
 - Maximum of 4 data wires for support of 8 channels
 - Capability up to 192k/32-bit input.
 - Capability up to 384k/32-bit output.
- I2S6 – ADC/DAC interface
 - Support standard I2S in **Master input**
 - Maximum of 1 data wire for 2-channel support
 - Capability up to 192k/32-bit input.
 - Capability for up to 384k/32-bit output.
- MCLK Support (I2S0/I2S1/I2S2/I2S4/I2S6)
 - Synced MCLK and I2S BCK
 - Capability up to 256fs in 192k mode
 - Capability up to 512fs in 96k mode
 - Capability up to 1024fs in 48k mode
 - Do not support 192fs or 384fs related

IP	Pin Name
I2S0	I2SIN0_LRCK / I2SIN0_BCK / I2S0_MCK / I2SIN0_DI/ I2SOUT0_DO
I2S1	I2SIN1_LRCK / I2SIN1_BCK / I2S1_MCK / I2SIN1_DI/ I2SOUT1_DO
I2S2	I2SIN2_LRCK / I2SIN2_BCK / I2SIN2_DI/ I2SOUT2_DO
I2S4	I2SIN4_LRCK / I2SIN4_BCK / I2SIN4_DI0/ I2SOUT4_DO0 / I2SIN4_DI1/ I2SOUT4_DO1 / I2SIN4_DI2/ I2SOUT4_DO2 / I2SIN4_DI3/ I2SOUT4_DO3
I2S6	I2SIN6_LRCK / I2SIN6_BCK / I2SIN6_DI/ I2SOUT6_DO
PMIC DL Interface	AUD_CLK_MOSI AUD_DAT_MOSI0 AUD_DAT_MOSI1
PMIC UL Interface	AUD_CLK_MISO AUD_DAT_MISO0 AUD_DAT_MISO1

Introducing an application for I2S connectivity.

MediaTek has a highly flexible design between HW functions (IP: intellectual property) and Pad.

A relationship between IP and pad can use AUX_FUNC to select Pad connect to which IP.

Table 1-1. GPIO function

Aux Func.0	Aux Func.1	Aux Func.2	Aux Func.3	Aux Func.4	Aux Func.5	Aux Func.6	Aux Func.7
B:GPIO15	O:I2SIN0_BCK	-	-	-	I1:ADSP_JTAG0_TRS TN	I0:SCP_JTAG_LITTLE _TRSTN_VCORE	I0:CONN_BGF_MCU _TRST_B
B:GPIO16	O:I2SIN0_LRCK	-	-	-	I0:ADSP_JTAG0_TCK	I1:SCP_JTAG_LITTLE _TCK_VCORE	I0:CONN_BGF_MCU _TCK
B:GPIO17	I0:I2SIN0_DI	-	-	-	I1:ADSP_JTAG0_TMS	I1:SCP_JTAG_LITTLE _TMS_VCORE	I1:CONN_BGF_MCU _TMS

Aux Func.0	Aux Func.1	Aux Func.2	Aux Func.3	Aux Func.4	Aux Func.5	Aux Func.6	Aux Func.7
B:GPIO18	O:I2SOUT0_DO	-	-	-	O:ADSP_JTAG0_TDO	O:SCP_JTAG_LITTLE_TDO_VCORE	O:CONN_BGF_MCU_TDO
B:GPIO19	O:I2SIN1_BCK	-	-	-	I1:ADSP_JTAG0_TDI	I1:SCP_JTAG_LITTLE_TDI_VCORE	I0:CONN_BGF_MCU_TDI
B:GPIO20	O:I2SIN1_LRCK	-	I0:MFG_EB_JTAG_TRSTN	-	I1:ADSP_JTAG1_TRSTN	I0:SCP_JTAG0_TRSTN_VCORE	I0:CONN_WF_MCU_TRST_B
B:GPIO21	I0:I2SIN1_DI	-	I0:MFG_EB_JTAG_TCK	-	I0:ADSP_JTAG1_TCK	I1:SCP_JTAG0_TCK_VCORE	I0:CONN_WF_MCU_TCK
B:GPIO22	O:I2SOUT1_DO	-	I0:MFG_EB_JTAG_TMS	-	I1:ADSP_JTAG1_TMS	I1:SCP_JTAG0_TMS_VCORE	I1:CONN_WF_MCU_TMS
B:GPIO23	O:I2SIN2_BCK	O:I2SIN0_MCK	O:MFG_EB_JTAG_TDO	I0:DSI1_TE	O:ADSP_JTAG1_TDO	O:SCP_JTAG0_TDO_VCORE	O:CONN_WF_MCU_TDO
B:GPIO24	O:I2SIN2_LRCK	O:I2SIN1_MCK	I0:MFG_EB_JTAG_TDI	O:LCM1_RST	I1:ADSP_JTAG1_TDI	I1:SCP_JTAG0_TDI_VCORE	I0:CONN_WF_MCU_TDI
B:GPIO25	I0:I2SIN2_DI	O:BPI_BUS20	I0:DSI2_TE	O:CONN_BPI_BUS20	O:ANT_SEL20	-	-
B:GPIO26	O:I2SOUT2_DO	O:BPI_BUS21	O:LCM2_RST	O:CONN_BPI_BUS21	O:ANT_SEL21	-	-
B:GPIO113	O:MIPI4_D_SCLK	I0:DSI1_TE	-	O:DMIC4_CLK	-	O:MD_GPS_L1_BLA_NK	-
B:GPIO114	B0:MIPI4_D_SDATA	O:LCM1_RST	-	I0:DMIC4_DAT	-	O:MD_GPS_L5_BLA_NK	-
B:GPIO109	O:MIPI2_D_SCLK	O:CONN_MIPI2_SCLK	O:DMIC1_CLK	-	-	--	-
B:GPIO110	B0:MIPI2_D_SDATA	B0:CONN_MIPI2_SDATA	I0:DMIC1_DAT	-	-	-	-
B:GPIO107	O:MIPI1_D_SCLK	O:CONN_MIPI1_SCLK	O:DMIC3_CLK	-	-	-	-
B:GPIO108	B0:MIPI1_D_SDATA	B0:CONN_MIPI1_SDATA	I0:DMIC3_DAT	-	-	-	-
B:GPIO63	-	-	O:BPI_BUS22	O:SPI5_B_CLK	O:I2SOUT4_DATA3	O:CONN_BPI_BUS22	-
B:GPIO4	-	-	B0:SPI7_A_MI	O:SPI5_B_CSB	O:I2SIN4_LRCK	O:CONN_TCXOENAREQ	O:DBG_MON_A20
B:GPIO62	-	-	B0:SPI7_A_MO	-	O:I2SOUT4_DATA0	O:WIFI_TXD	O:DBG_MON_A21
B:GPIO137	B1:SCL0	-	O:SPI7_A_CLK	-	O:I2SOUT4_DATA1	O:MD32_0_GPIO0	O:DBG_MON_A22
B:GPIO138	B1:SDA0	-	O:SPI7_A_CSB	-	O:I2SOUT4_DATA2	O:MD32_1_GPIO0	O:DBG_MON_A23
B:GPIO65	-	O:CMFLASH0	-	O:USB_DRVVBUS	O:I2SIN4_BCK	-	O:DBG_MON_A24
B:GPIO5	-	O:CMFLASH1	B0:SPI5_A_MO	I1:IDDIG	I0:I2SIN4_DATA0	-	O:DBG_MON_A25
B:GPIO6	-	O:CMFLASH2	B0:SPI5_A_MI	I0:VBUSVALID	I0:I2SIN4_DATA1	-	O:DBG_MON_A26
B:GPIO151	B1:SCL7	O:CMFLASH3	O:SPI5_A_CLK	-	I0:I2SIN4_DATA2	I1:MD32_0_RXD	O:DBG_MON_A27
B:GPIO152	B1:SDA7	-	O:SPI5_A_CSB	-	I0:I2SIN4_DATA3	O:MD32_0_TXD	O:DBG_MON_A28
B:GPIO33	B1:KPROW0	O:I2SIN0_MCK	O:BPI_BUS22	O:CONN_BPI_BUS22	-	-	O:DBG_MON_A29
B:GPIO34	B1:KPROW1	O:I2SIN1_MCK	-	-	-	-	-
B:GPIO94	O:BPI_BUS9	O:CONN_BPI_BUS9	-	O:PWM_3	O:AUD_DAC_26M_CLK	-	O:DBG_MON_A9
B:GPIO95	O:BPI_BUS10	O:CONN_BPI_BUS10	-	-	O:I2SOUT4_DATA0	-	O:DBG_MON_A10
B:GPIO96	O:BPI_BUS11	O:CONN_BPI_BUS11	-	-	O:I2SOUT4_DATA1	-	O:DBG_MON_A11
B:GPIO97	O:BPI_BUS12	O:CONN_BPI_BUS12	B0:SPI6_B_MI	-	O:I2SOUT4_DATA2	-	O:DBG_MON_A12
B:GPIO98	O:BPI_BUS13	O:CONN_BPI_BUS13	O:SPI6_B_CLK	-	O:I2SOUT4_DATA3	-	O:DBG_MON_A13
B:GPIO99	O:BPI_BUS14	O:CONN_BPI_BUS14	O:SPI6_B_CSB	-	O:I2SIN4_BCK	-	O:DBG_MON_A14
B:GPIO100	O:BPI_BUS15	O:CONN_BPI_BUS15	B0:SPI6_B_MO	-	I0:I2SIN4_DATA0	-	O:DBG_MON_A15
B:GPIO101	O:BPI_BUS16	O:CONN_BPI_BUS16	O:CLKM0	-	I0:I2SIN4_DATA1	-	O:DBG_MON_A16
B:GPIO102	O:BPI_BUS17	O:CONN_BPI_BUS17	O:CLKM1	-	I0:I2SIN4_DATA2	-	O:DBG_MON_A17
B:GPIO103	O:BPI_BUS18	O:CONN_BPI_BUS18	O:CLKM2	O:DMIC1_CLK	I0:I2SIN4_DATA3	-	O:DBG_MON_A18
B:GPIO104	O:BPI_BUS19	O:CONN_BPI_BUS19	O:CLKM3	I0:DMIC1_DAT	O:I2SIN4_LRCK	-	O:DBG_MON_A19
B:GPIO153	B1:SCL8	O:I2SIN1_BCK	-	-	O:UDI_TDO_4	I0:MFG_TSFDC_26M	-
B:GPIO154	B1:SDA8	O:I2SIN1_LRCK	-	-	O:UDI_TDO_5	I0:MFG_TSFDC_SCF	-
B:GPIO125	O:CMMCLK2	I0:I2SIN1_DI	-	-	I0:UDI_TDI_6	-	-
B:GPIO117	-	O:I2SOUT1_DO	I1:URXD2	O:CLKM2	I0:UDI_TDI_2	O:PWM_2	-

Aux Func.0	Aux Func.1	Aux Func.2	Aux Func.3	Aux Func.4	Aux Func.5	Aux Func.6	Aux Func.7
B:GPIO161	B1:SCL12	O:I2SIN2_BCK	I1:UCTS2	-	O:CMVREF3	I0:MFG_TSFDC_SCK	-
B:GPIO162	B1:SDA12	O:I2SIN2_LRCK	O:URTS2	-	O:CMVREF2	I0:MFG_TSFDC_SDI	-
B:GPIO129	O:CMMCLK6	I0:I2SIN2_DI	-	-	O:CMVREF1	-	-
B:GPIO121	-	O:I2SOUT2_DO	O:UTXD2	O:PWM_2	O:CMVREF0	-	-
B:GPIO78	B0:MSDC2_CLK	O:I2SIN6_0_BCK	B0:TP_GPIO11_AO	I0:CONN_BGF_DSP_L1_JCK	I1:SCP_ITAG0_TCK_VLP	I1:SSPM_ITAG_TCK_VLP	I0:IO_ITAG_TCK
B:GPIO79	B1:MSDC2_CMD	O:I2SIN6_0_LRCK	B0:TP_GPIO12_AO	I1:CONN_BGF_DSP_L1_JMS	I1:SCP_ITAG0_TMS_VLP	I1:SSPM_ITAG_TMS_VLP	B1:IO_ITAG_TMS
B:GPIO80	B1:MSDC2_DAT0	I0:I2SIN6_0_DI	B0:TP_GPIO13_AO	I0:CONN_BGF_DSP_L1_JDI	I1:SCP_ITAG0_TDI_VLP	I1:SSPM_ITAG_TDI_VLP	I1:IO_ITAG_TDI
B:GPIO83	B1:MSDC2_DAT3	O:I2SOUT6_0_DO	O:PWM_VLP	I0:MD_INT3	B0:TP_GPIO4_AO	O:SRCLKENA1	I0:SRCLKENAIO
B:GPIO159	B1:SCL11	O:I2SIN0_BCK	O:BPI_BUS16	O:ANT_SEL16	O:CMVREF4	O:MFG_TSFDC_TSSEL0	-
B:GPIO160	B1:SDA11	O:I2SIN0_LRCK	O:BPI_BUS17	O:ANT_SEL17	O:CMVREF5	O:MFG_TSFDC_RCK_SELB	-
B:GPIO128	O:CMMCLK5	I0:I2SIN0_DI	O:BPI_BUS18	O:ANT_SEL18	O:CMVREF6	-	-
B:GPIO120	O:PWM_1	O:I2SOUT0_DO	O:BPI_BUS19	O:ANT_SEL19	O:CMVREF7	-	-
B:GPIO184	O:ANT_SEL11	O:I2SIN6_0_BCK	I0:PTA_EXT_FREQ	O:BPI_BUS11	O:CMVREF4	O:CLKM0	-
B:GPIO185	O:ANT_SEL12	O:I2SIN6_0_LRCK	I0:PTA_EXT_ACT	O:BPI_BUS12	O:CMVREF5	O:CLKM1	-
B:GPIO186	O:ANT_SEL13	I0:I2SIN6_0_DI	I0:PTA_EXT_PRI	O:BPI_BUS13	O:CMVREF6	O:CLKM2	-
B:GPIO187	O:ANT_SEL14	O:I2SOUT6_0_DO	O:PTA_EXT_WLAN_ACT	O:BPI_BUS14	O:CMVREF7	O:CLKM3	-

1.4.1.1 I2S Application

Table 1-2. I2S application

Interface	Direction	Suggestion: I2S IF assignment
I2S0	Input/Output	Reserved, for 3 rd party
I2S1	Input/Output	Reserved, for 3 rd party
I2S2	Input/Output	Reserved, for 3 rd party
I2S4	Input/Output	External DSP, SmartPA with internal DSP
I2S6	Input/Output	Reserved for 3rd party
MTK proprietary Audio IF	Input*4 Output*4	ADC *4, for Main Mic, Ref Mic, 3 rd Mic, 4 th Mic, DMIC*4 DAC *4, for receiver, headset L, headset R, ext amp

For the other applications, consult with the MTK ACS team for the best I2S IF assignment.

The I2S4 should be used for the smart PA share pin application.

1.5 Mediatek Aurisys and Open DSP

1.5.1 Overview

The requirement for sound quality enhancements that add listening pleasure has been increasing. To enrich the listening experience of using mobile devices, there is an increasing number of sound processing solutions provided by vendors.

Aurisys is introduced to facilitate the sound processing solution development and use of MTK platforms. Aurisys is a framework constructed upon Android Audio Framework. It includes standardized interfaces for sound processing and tuning, integrated DSP sound subsystem, and software debug interface. This concept of Aurisys will be briefly described in this document.

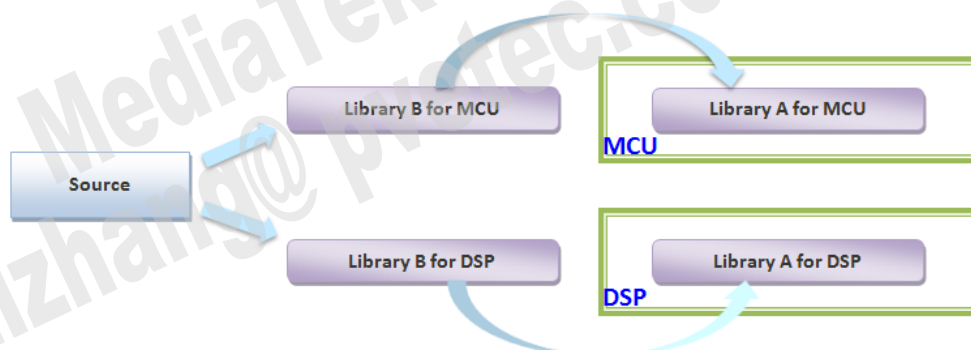


Figure 1-12. Aurisys concept

1.5.1.1 Aurisys Structure

As depicted in Figure 1-13, the Aurisys structure contains Scene Handler, Library Manager, integrated DSP framework, Modem/Audio HW subsystems, and [standardized](#) software interfaces:

- Aurisys Scene Handler (ARSH)
 - The Aurisys Scene Handler serves as the middleware between the audio system and the sound processing IPs. It is created scene by scene. For example, the Aurisys Playback Handler is used for playback effects in the scene of playback. It provides interfaces to call the corresponding IPs and manages them.
- Aurisys Library Manager (ARLM)
 - The Aurisys Library Manager maintains the library information for each Aurisys Scene Handler, including the list and status of sound IPs.
- Aurisys Software Interface (ARSI)
 - ARSI is the abbreviation of Aurisys Software Interface. It is provided for interfacing with sound enhancing tasks. To process sounds, a unified interface which is portable between MTK platforms is provided. The interface to parse and transmit parameters between PC tool, APMCU, and DSP is also included. The interface is designed for ease of use while being general enough so that new algorithms can be easily added to the existing frameworks. The difference between ARSI and ARSH is that ARSI provides an interface between ARSH and sound IPs; while ARSH provides an interface for Android Audio HAL to request processing sounds.
- Modem
 - The term 'modem' refers to the modem IC. The MTK internal chipset contains a DSP for processing voice enhancement and voice codec. In the Aurisys structure, we disable the voice enhancement but reserve the codec in the modem IC so that we can simply add algorithms in the integrated open DSP on the application side.
- Audio HW
 - It refers to the interface between processors and hardware devices for transferring sound data. The devices include speakers, microphones, earphones, Bluetooth devices, USB devices, etc.

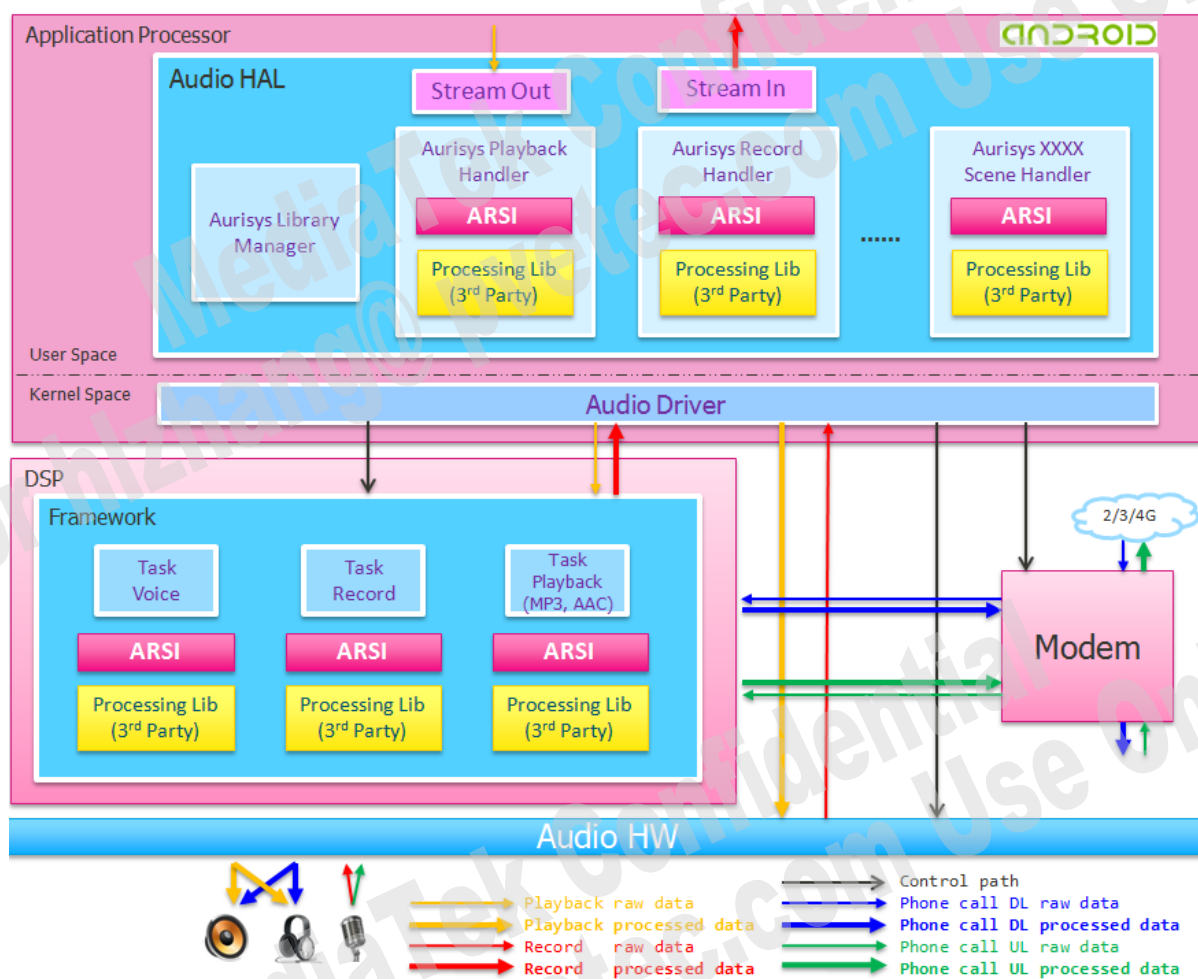


Figure 1-13. Aurisys structure

1.5.2 Data Path Customization Guideline

When using DSP for audio processing and output, it is necessary to configure the mix routing path and channel configuration information for each audio stream in the DSP. The public DSP task configuration provides a basic reference configuration, but in actual scenarios, some modifications to each DSP task are required. This section mainly introduces the configuration file modifications involved in the customization of ADSP-related functions.

1.6 Kernel DTS Configuration

In the DTS, the attribute configuration related to DSP audio streams is shown as Figure 1-14:

```

&snd_audio_dsp-{
    mtk-dsp-call-final = <0x0 0xe 0x13 0x12 0x18000>;
    mtk-dsp-hfp-client-rx = <0x5 0xffffffff 0x12 0xffffffff 0x20000>;
    mtk-dsp-anc = <0x5 0xffffffff 0x10 0xffffffff 0x20000>;
    mtk-dsp-extstream1 = <0x5 0xffffffff 0xf 0xffffffff 0x20000>;
    mtk-dsp-extstream2 = <0x5 0xffffffff 0x11 0xffffffff 0x20000>;
    mtk-dsp-sub-playback = <0x1 0x0 0xffffffff 0xffffffff 0x30000>;
    mtk-dsp-playback0 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback1 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback2 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback3 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback4 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback5 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback6 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback7 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback8 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback9 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback10 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback11 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback12 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback13 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback14 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback15 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-capture-mch = <0x1 0xffffffff 0x18 0xffffffff 0x20000>;
    swdsp-smartpa-process-enable = <0x0>;
};

```

Figure 1-14. DTS feature list

Each DSP task attribute node saves a five-digit array, where the array sequence represents the enable, dl_mem, ul_mem, ref_mem, and the size of the shared memory used by the DSP task. Among all attributes, the user needs to configure the dl_mem (downlink), ul_mem (uplink), and ref_mem (reference signal) according to the actual usage of memif by each DSP task. For example, for the mtk-dsp-sub-playback task, the dl output memif used is DL0, then the corresponding dl_mem is filled with 0x0. Most tasks (such as playback0) do not actually use hw memif, so they need to be configured as 0xffffffff. The enumeration type of the memif node is defined in mt6897-afe-common.h.

```

enum {
    MT6991_MEMIF_DL_4CH,
    MT6991_MEMIF_DL_24CH,
    MT6991_MEMIF_VUL0,
    MT6991_MEMIF_VUL1,
    MT6991_MEMIF_VUL2,
    MT6991_MEMIF_VUL3,
    MT6991_MEMIF_VUL4,
    MT6991_MEMIF_VUL5,
    MT6991_MEMIF_VUL6,
    MT6991_MEMIF_VUL7,
    MT6991_MEMIF_VUL8,
    MT6991_MEMIF_VUL9,
    MT6991_MEMIF_VUL10,
    MT6991_MEMIF_VUL24,
    MT6991_MEMIF_VUL25,
    MT6991_MEMIF_VUL26,
    MT6991_MEMIF_VUL_CM0,
    MT6991_MEMIF_VUL_CM1,
    MT6991_MEMIF_VUL_CM2,
    MT6991_MEMIF_ETDM_IN0,
    MT6991_MEMIF_ETDM_IN1,
    MT6991_MEMIF_ETDM_IN2,
    MT6991_MEMIF_ETDM_IN3,
    MT6991_MEMIF_ETDM_IN4,
    MT6991_MEMIF_ETDM_IN6,
    MT6991_MEMIF_HDMI,
    MT6991_MEMIF_NUM,
};

```

Figure 1-15. memif define

It should be noted that after modifying the node used by the corresponding task in the DTS, the PCM used by the corresponding task in the HAL also needs to be modified. For the dsp-playback task, the mPlaybackUIndex and other variables in the openPlaybackTask function of AudioDspStreamManager.cpp in the audio HAL need to be modified to select the corresponding dl playback stream. The number of channels output by the DSP is also modified in this function, which is shown as Figure 1-16:

```

.../*get card index and pcm index*/
.../*todo::pcm should move to platform*/
...mStreamCardIndex = AudioALSADeviceParser::getInstance()->GetCardIndexByString(keypcmPlayback11);
...mDspIndex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmPlaybackDsp);

...mPlaybackULindex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmCapture6);
...mPlaybackIVindex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmCapture4);
...mPlaybackDLindex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmPlayback11);

```

Figure 1-16. Audio hal pcm config

The architecture of each task under DSP is shown as Figure 1-17:

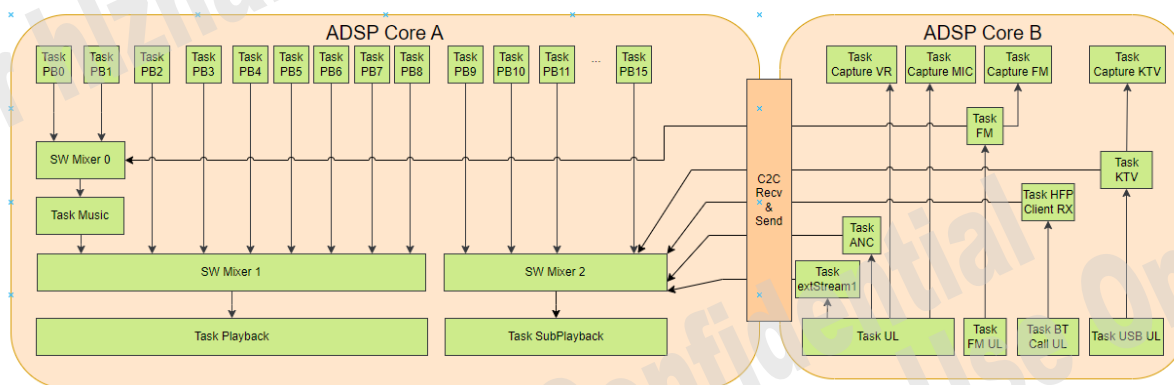


Figure 1-17. ADSP architecture

1.6.1 Audio Path Configuration(AFE HW)

Before playing and recording through the audio DSP PCM stream, the corresponding AFE hardware path needs to be configured. The configuration process follows the standard process of tinysalsa, configuring the corresponding kcontrol path through tinymix, and opening and starting the corresponding hw path. Figure 1-18 shows the playback path of DL3->ADDA.

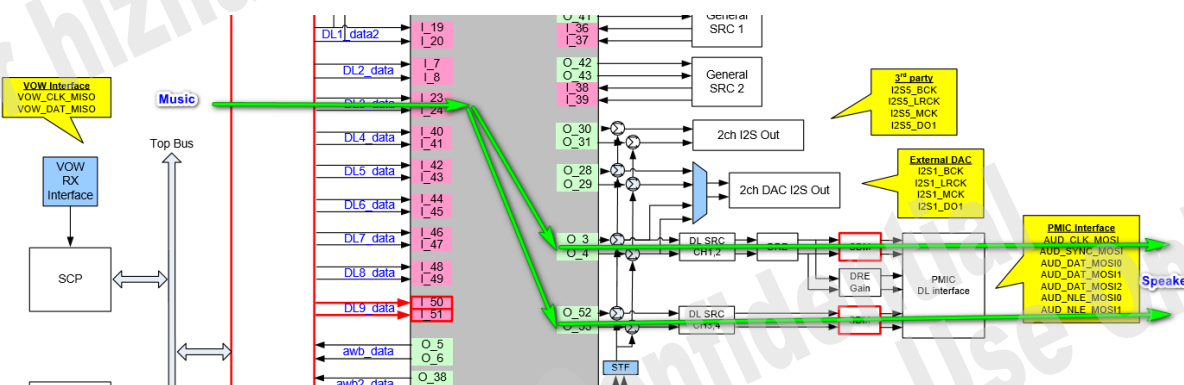


Figure 1-18. Audio path for playing music

The corresponding configuration in audio_device.xml is shown as Figure 1-19:

```

<path name="PLAYBACK3_TO_ADDA_DL" value="turnon">
  <ctl name="ADDA_DL_CH1 DL3_CH1" value="1" />
  <ctl name="ADDA_DL_CH2 DL3_CH2" value="1" />
  <ctl name="ADDA_DL_CH3 DL3_CH1" value="1" />
  <ctl name="ADDA_DL_CH4 DL3_CH2" value="1" />
</path>
<path name="PLAYBACK3_TO_ADDA_DL" value="turnoff">
  <ctl name="ADDA_DL_CH1 DL3_CH1" value="0" />
  <ctl name="ADDA_DL_CH2 DL3_CH2" value="0" />
  <ctl name="ADDA_DL_CH3 DL3_CH1" value="0" />
  <ctl name="ADDA_DL_CH4 DL3_CH2" value="0" />
</path>
<path name="headphone_output" value="turnon">
  <ctl name="DAC In Mux" value="Normal Path" />
  <ctl name="HPL Mux" value="Audio Playback" />
  <ctl name="HPR Mux" value="Audio Playback" />
</path>
<path name="headphone_output" value="turnoff">
  <ctl name="HPL Mux" value="Open" />
  <ctl name="HPR Mux" value="Open" />
</path>

```

连通DL3 <==> PMIC

断开DL3 <==> PMIC

PMIC 6359 -> Speaker

Figure 1-19. Playback_3 to PMIC

1.6.2 Audio Path Configuration (INT ADSP)

There are multiple tasks (bus out) in Auto ADSP, and each task needs to configure different algorithm processing parameters according to the needs and actual scenarios, such as channel in, channel out, and whether to bypass the algorithm. At the same time, it is generally necessary to map the channels of each bus to the corresponding channel positions in the final output TDM format, that is, to configure the channel map information. Therefore, the audio_dsp_config.xml configuration is provided to modify the various attributes and mix paths of the auto DSP task. The general architecture of tasks in ADSP is shown as Figure 1-20:

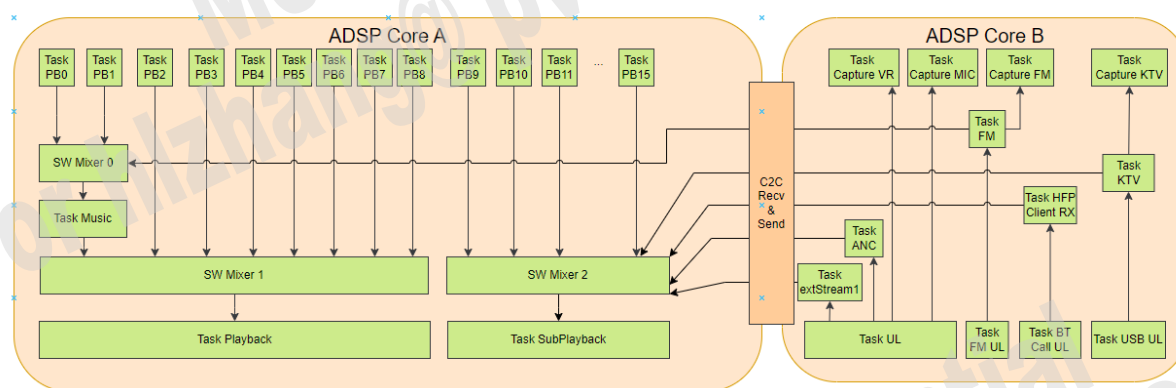


Figure 1-20. ADSP architecture

1.6.2.1 XML Configuration Instructions

Configuration XML path

The path of the XML on the device is as follows: vendor/etc/audio_dsp_config.xml

In the code, the path is as follows: device/<project_name>/audio_dsp_config.xml

1.6.2.2 mixer_target & mixer_source

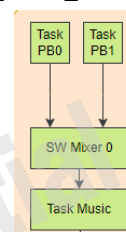
Within the XML, each DSP task is categorized into `sw_mixer_target` and `sw_mixer_source` based on its placement in the DSP mixer. For Task Music, it functions as both a `sw_mixer_target` and a `sw_mixer_source`.

- `sw_mixer_target` Corresponds to the output end of the `sw_mixer`, such as Task Music, Task Playback, and Task SubPlayback
- `sw_mixer_source` Corresponds to the input end of the `sw_mixer`, such as Task PB0 (name="`sw_mixer_playback0`")

The role of the task as a target or source is predetermined and cannot be configured, but the `sw_mixer_source` can be configured to mix into the desired target in the XML. The corresponding modification method is to write the `sw_mixer_source` node with the corresponding name into the node of the `sw_mixer_target` in the XML, as shown in the example below, indicating that task PB0 (`sw_mixer_playback0`) will mix into task music (`sw_mixer_music`).

Ex1:

```
<sw_mixer_target name="sw_mixer_music" ch_in="2"
  ch_out="16" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
      ch_map="0x5555,0xaaaa"/>
  </sw_mixer_source>
</sw_mixer_target>
```



Currently, the configurable `sw_mixer_sources` are: `sw_mixer_playback0` ~ `sw_mixer_playback15`, `sw_mixer_FM_ADSP`, `sw_mixer_hfp_client_rx`, `sw_mixer_anc`, `sw_mixer_extstream1`, `sw_mixer_extstream2`, `sw_mixer_ktv`.

1.6.2.3 Task Attribute

In each DSP task, different algorithm input and output channels, and channel map information will be configured as required by the scenario. In the XML, the following attributes can be configured for each task:

- For `sw_mixer_target`
 - `ch_in`: Algorithm input channel for the task
 - `ch_out`: Algorithm output channel for the task
 - `aurisys_on`: Whether to enable the task algorithm; it can only be configured as 0 when `ch_in` = `ch_out`

For example, in the example below, the algorithm input channel of task music is 2, the algorithm processed output channel is 16, and the algorithm is not bypassed.

```
<sw_mixer_target name="sw_mixer_music" ch_in="2" ch_out="16" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
      ch_map="0x5555,0xaaaa"/>
  </sw_mixer_source>
</sw_mixer_target>
```

- For `sw_mixer_source`, each profile has the following attributes:
 - `ch_in`: Algorithm input channel for the task
 - `ch_out`: Algorithm output channel for the task
 - `aurisys_on`: Whether to enable the task algorithm, can only be configured as 0 for `pb0`~`pb15` when `ch_in` = `ch_out`
 - `ch_map`: Channel map information when the source is mapped to the `sw_mixer_target`, which is an array. The index in the array represents the channel index of the source, and the value is a 16-bit value, each bit representing whether to map to the corresponding channel of the target. For example, `ch_map="0x5555,0xaaaa"`, `ch_map[0]` =

0x5555, representing the mapping value of source channel 1 is 0x5555, corresponding to binary 0101010101010101, which maps to channels 1, 3, 5, 7, 9, 11, 13,

For source tasks, the input channels are typically provided by the data source, such as audiofinger, and may consist of any number of channels. Therefore, in the source task, the XML can be configured to support multiple profiles for different ch_in settings, allowing the selection of different ch_out and ch_map settings based on the ch_in. For example, the task pb14 has two profiles: if the channel input to adsp for task pb14 is 2 channels, the first profile is used; if it is 12 channels, the second profile is used.

```
<sw_mixer_source name="sw_mixer_playback14">
  <profile ch_in="2" ch_out="2" aurisys_on="0"
    ch_map="0x5555,0xaaaa"/>
  <profile ch_in="12" ch_out="12" aurisys_on="0"
    ch_map="0x1,0x2,0x4,0x8,0x10,0x20,0x40,0x80,0x100,0x200,0x400,0x800,0x1000,0x2000,0x4000,0x8000"/>
</sw_mixer_source>
```

For the music task, since it is both a source and a target, it takes the ch_in attribute from its target properties and finds the corresponding profile in the source to configure it accordingly.

1.6.2.4 Configuration Example

Here is an example of task pb0 configuration as follows:

```
<sw_mixer_target name="sw_mixer_music" ch_in="8" ch_out="12" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
      ch_map="0x4,0x8"/>
    <profile ch_in="4" ch_out="4" aurisys_on="0"
      ch_map="0x1,0x2,0x4,0x8 "/>
  </sw_mixer_source>
</sw_mixer_target>

<sw_mixer_target name="sw_mixer_playback" ch_in="16" ch_out="16" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_music">
    <profile ch_in="8" ch_out="12" aurisys_on="1"
      ch_map="0x1,0x2,0x4,0x8,0x10,0x20,0x40,0x80,0x100,0x200,0x400,0x800"
    >
  </sw_mixer_source>
</sw_mixer_target>
```

The corresponding data path process is as follows:

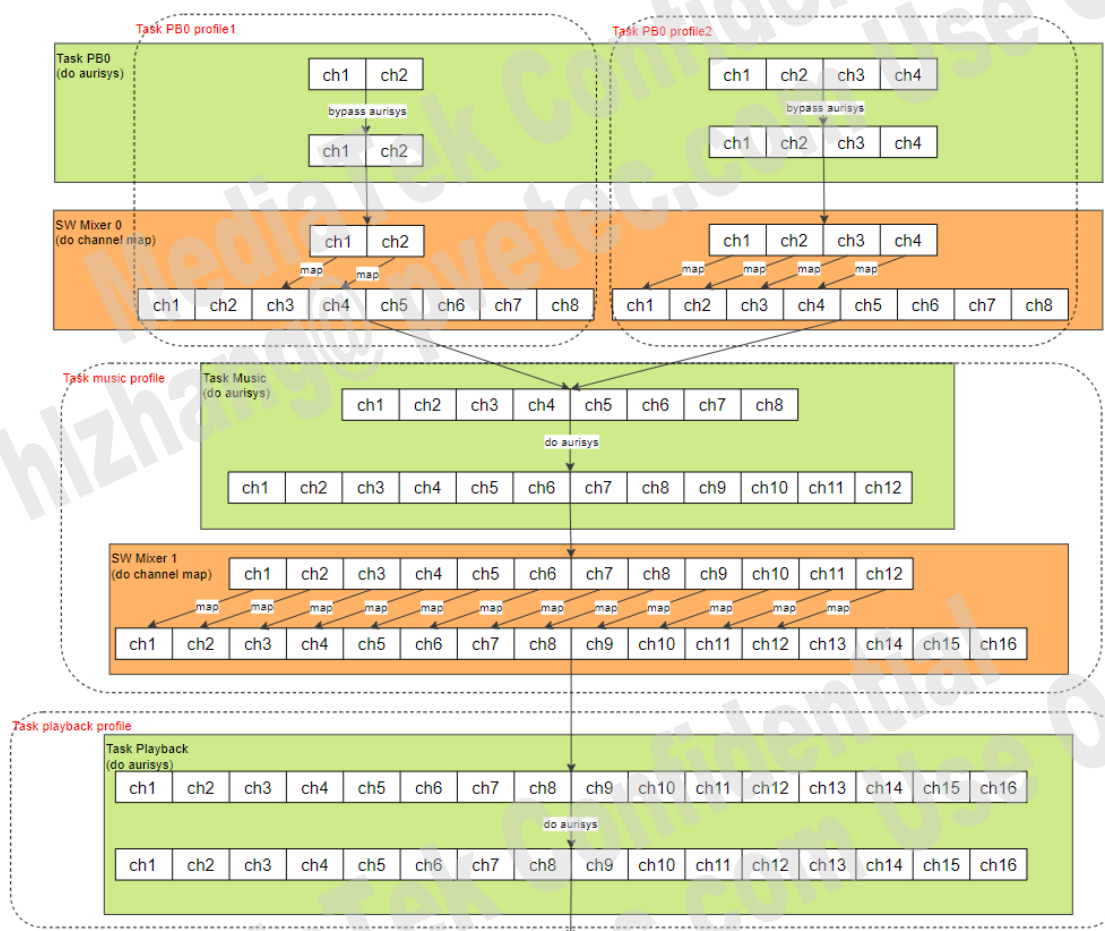


Figure 1-21. dsp task config

1.7 Summary

This chapter mainly introduces the Aurisys audio effect framework to facilitate users to integrate the required algorithms at the appropriate location. At the same time, in response to the customization of the audio bus scene, the configuration method of audio mix and channel map in ADSP is introduced.

Besides the above-mentioned features, other applications can also be carried out. However, considering the limited resource of DSP, the concurrent cases should be designed carefully. Please visit MediaTek Website and customer support Website (MediaTek On-Line, MOL) to get more information.

1.8 Appendix

1.8.1 MTK MOL

MTK provides a forum named MTK on-line to share FAQ, eCourse and the important announcement:

<http://online.mediatek.com>

You can type the keyword to search the related FAQ and eCourse.

You can explore the document tree to enter the related audio FAQ and eCourse.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.